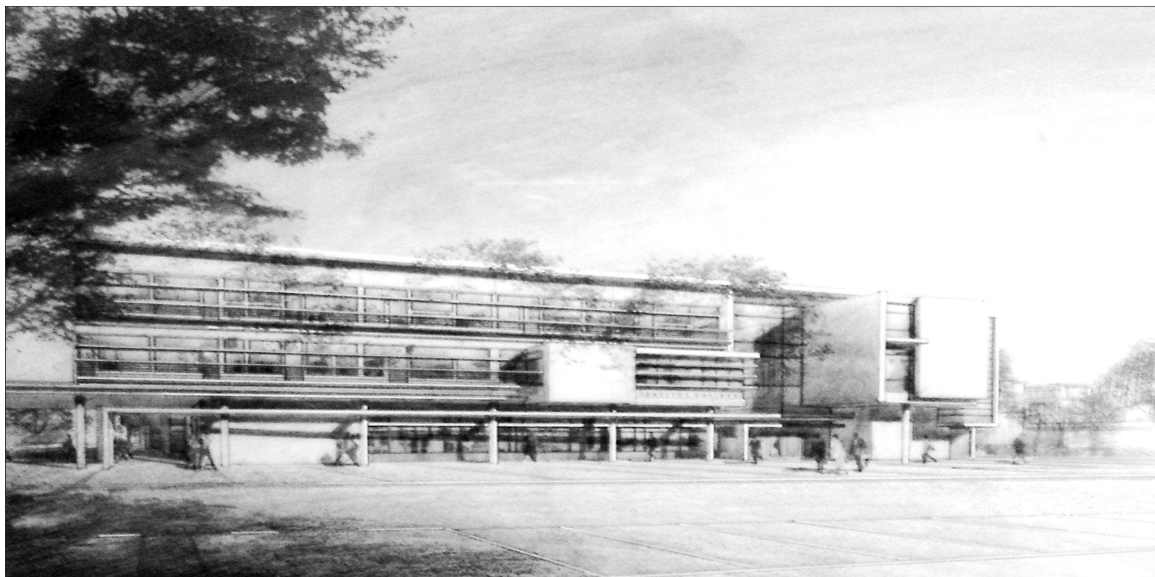




Licence SPI 2^{ème} année

MODULE MICROCONTRÔLEURS

TRAVAUX DIRIGES

OBJECTIFS GENERAUX

TD 1. ASPECTS MATERIELS 02

*Capacité mémoire,
 Analyse de **chronogrammes** de cycle bus,
 Démultiplexage de bus,
 Analyse d'un **décodage d'adresses** et **mapping mémoire**,
 Lecture, exploitation de schémas, **identification** et **repérage** de composants.*

TD 2. ASPECTS LOGICIELS 17

*Modes d'adressage du **HC11**,
 Implantation mémoire,
 Programmation en **assembleur**,
 Synthèse : écriture d'un **logiciel applicatif**.*

THIEBAUT Bernard

TD 1 : ASPECTS MATERIELS

1. CAPACITE MEMOIRE.

On dispose d'un mémoire **RAM** de **1 Mbits** constituée de cases mémoire de taille **1 octet** chacune, à accès « parallèle ».

1. Quel est le **nombre** de broches de **données** sur ce circuit ? Entrées ou sorties ?
2. Donner la **capacité** de cette mémoire en octets : valeur exacte puis arrondie en « unité Ko ».
3. Quel est le **nombre** de broches d'**adresses** sur ce circuit ? Entrées ou sorties ?
4. Intérêt d'une mémoire à accès « série ». Avantages et inconvénients.

2. DEMULTIPLEXAGE DE BUS.

On se propose dans cet exercice de réaliser le **démultiplexage** du bus d'adresses/données **AD0 à AD15** d'un microprocesseur du commerce.

1. Rappeler brièvement :

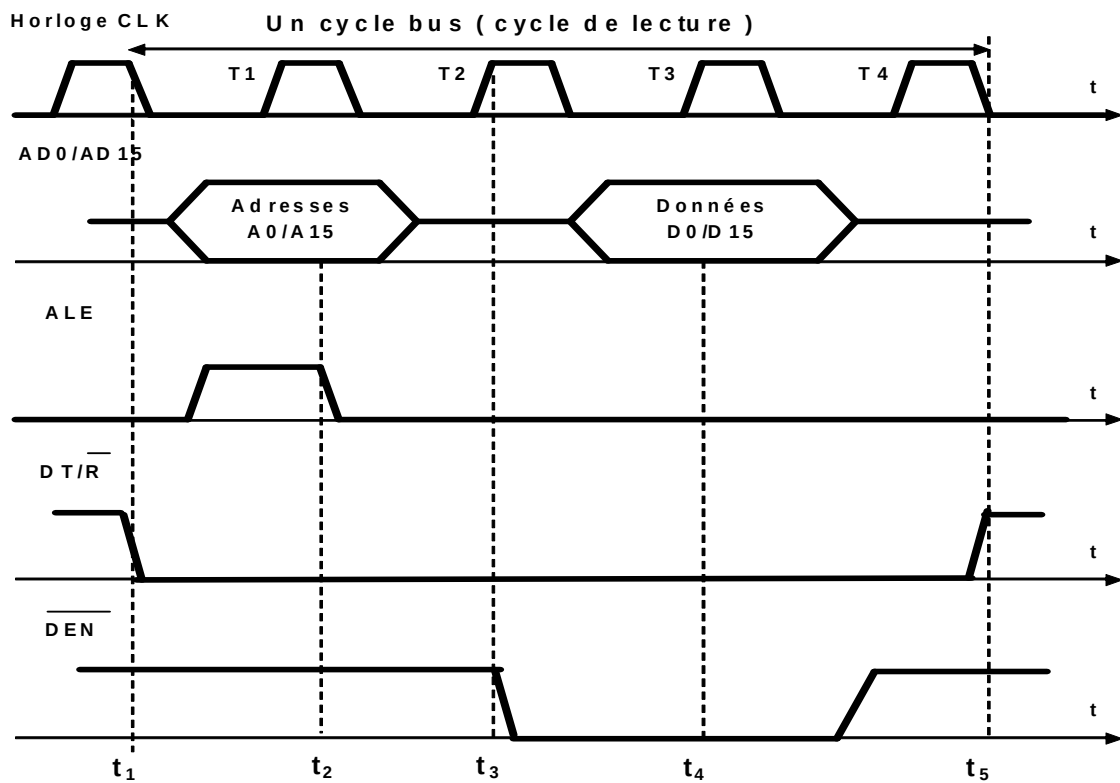
- la **définition** d'un bus,
- l'**intérêt** du **multiplexage** des adresses et des données,
- le **sens** (ou direction) de ce bus multiplexé,
- la **nécessité** du **démultiplexage** de ces deux bus pour avoir un fonctionnement correct.

Les signaux que l'on utilisera sont décrits ci-dessous :

- **AD0 à AD15** : le **bus multiplexé** (comme **A..dress D..ata**),
- **DT – R** : une **sortie** du microprocesseur indiquant :
au niveau **haut** : un transfert de données **vers l'extérieur** du microprocesseur (**D..ata T..ransmit**),
au niveau **bas** : un transfert de données **vers le** microprocesseur (**D..ata R..eceive**),
- **$\overline{DEN}$** une **sortie** du microprocesseur indiquant que le bus multiplexé véhicule des données (**$\overline{D..ata EN..able}$**),
- **ALE** une **sortie** du microprocesseur indiquant sur son **front descendant** que le bus multiplexé véhicule des adresses (**A..dress L..atch E..nable**).

La figure ci-après, réalisée à partir d'une documentation constructeur, fournit les chronogrammes simplifiés d'un **cycle de lecture** mémoire.

CHRONOGRAMMES SUR UN CYCLE DE LECTURE



2. Analyser ces chronogrammes et faites le lien avec les principaux signaux présentés ci-dessus.

3. On donne la documentation des 2 circuits suivants :

- le **transmetteur** de bus **8286**,
- l'**octal latch** **8282**.

Prendre connaissance de ces documentations, et en analysant les directivités des composants, indiquer sur quels bus il faut respectivement les relier.

Donner également, en fonction de la taille des bus, le nombre de **8286** et de **8282** nécessaires ici.

4. Proposer un montage utilisant ces composants pour **démultiplexer** le bus **AD0 / AD15**.

On pourra utiliser, en le complétant, le « canevas » de schéma proposé dans les pages suivantes.

Sur ce schéma dûment complété, on précisera bien les broches utilisées et on indiquera les endroits où le bus est multiplexé et ceux où il devient utilisable pour brancher d'autres composants type mémoire ou circuits périphériques d'E/S.

EXTRAIT DE LA DOCUMENTATION DU 8286

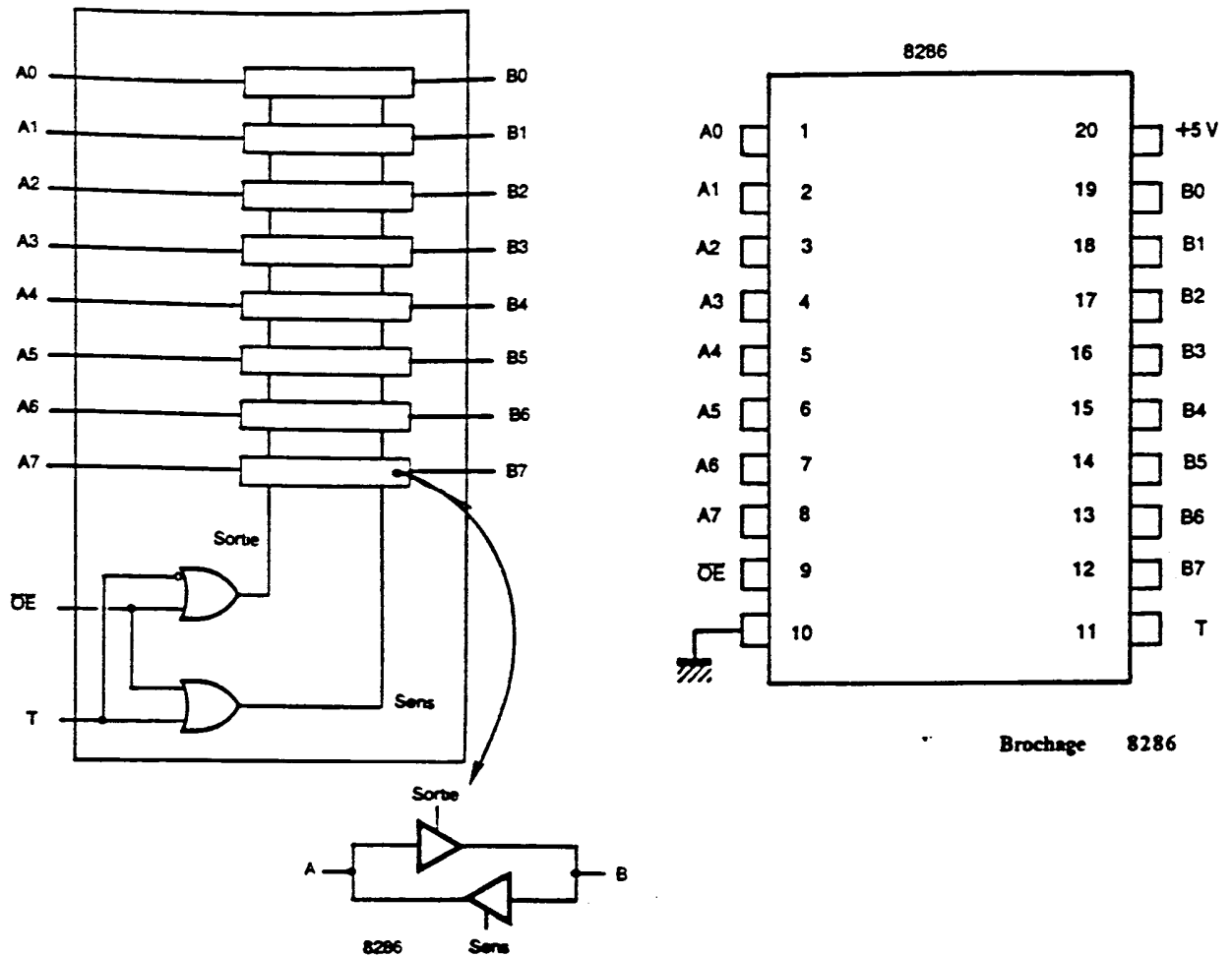
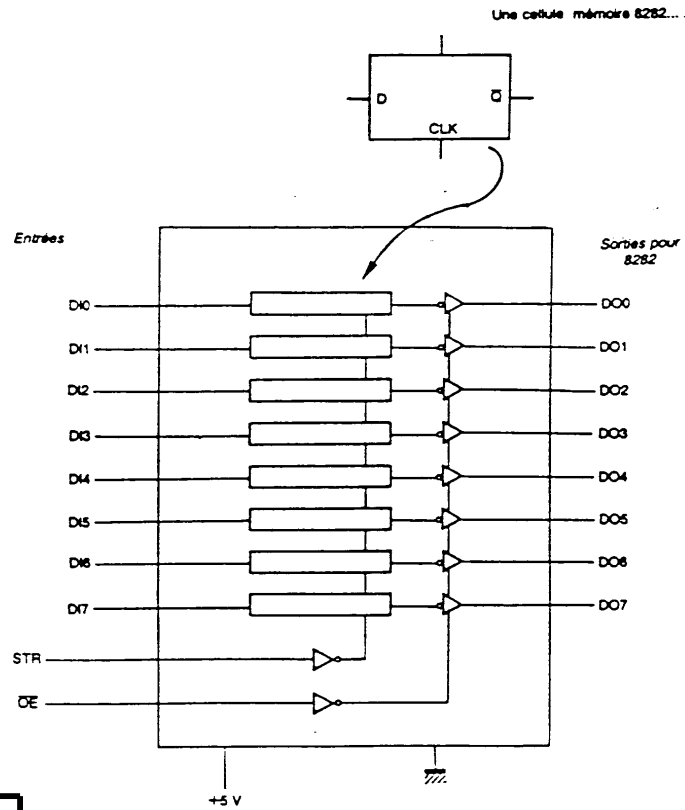


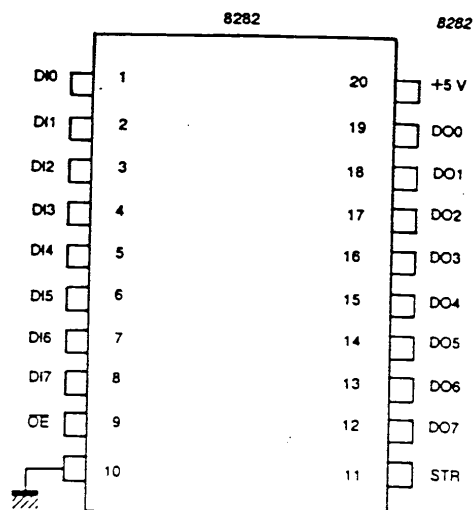
TABLE D'ETAT DU 8286

Niveaux sur		Lignes A	Lignes B	Signal sur la sortie réelle A ou B
OE	T			
Bas	Haut	Entrées	Sorties	Celui de l'entrée
Bas	Bas	Sorties	Entrées	Celui de l'entrée
Haut	Haut ou bas	Entrées ou sorties	Entrées ou sorties	Etat haute impédance

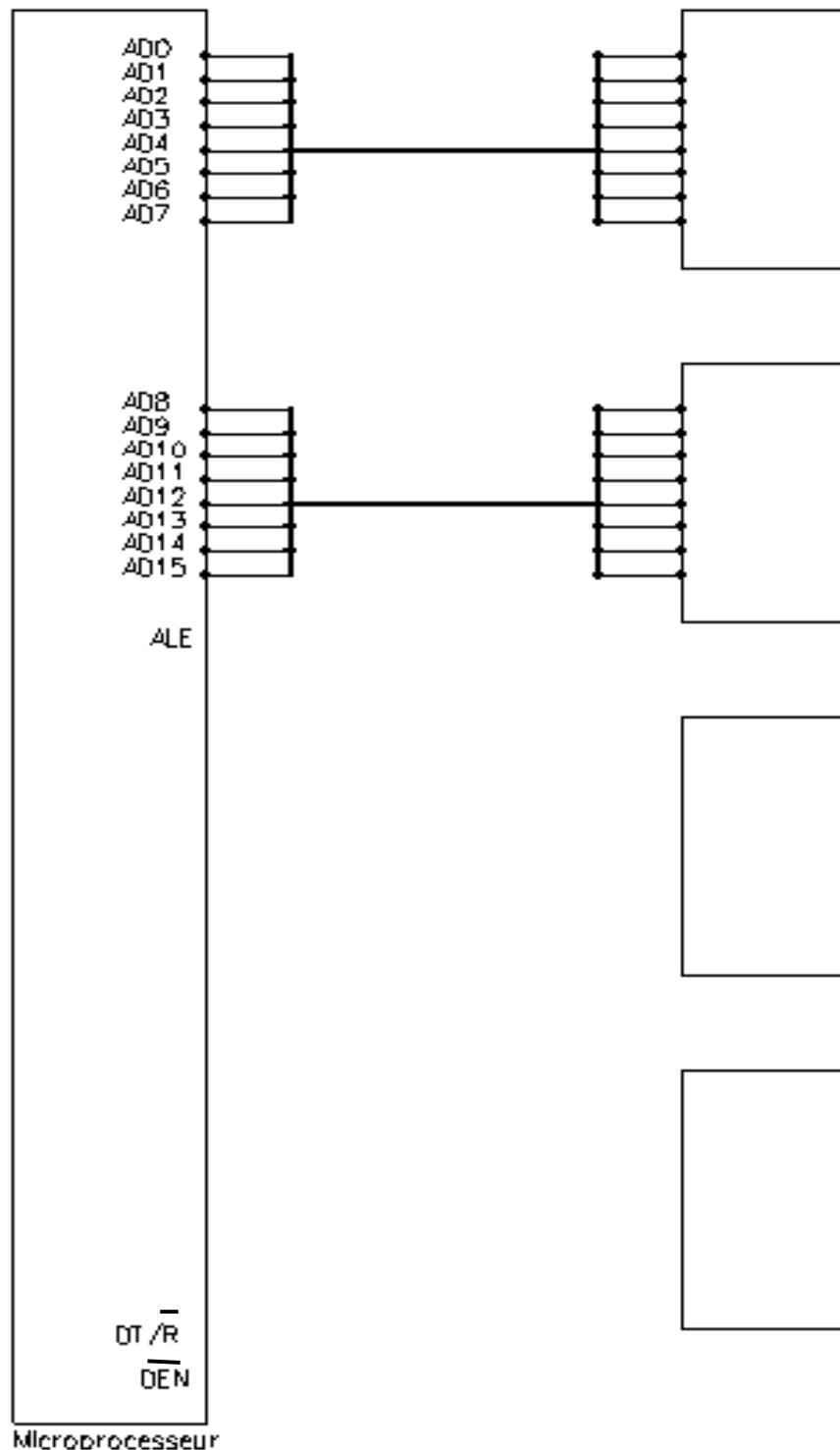
EXTRAIT DE LA DOCUMENTATION DU 8282



Sur front actif descendant de STR les DI_i sont mémorisés sur les DO_i
 Si STR = niveau haut, alors "transparence" du 8282



SCHEMA « CANEVAS » A COMPLETER



3. ANALYSE D'UN DECODAGE D'ADRESSES.

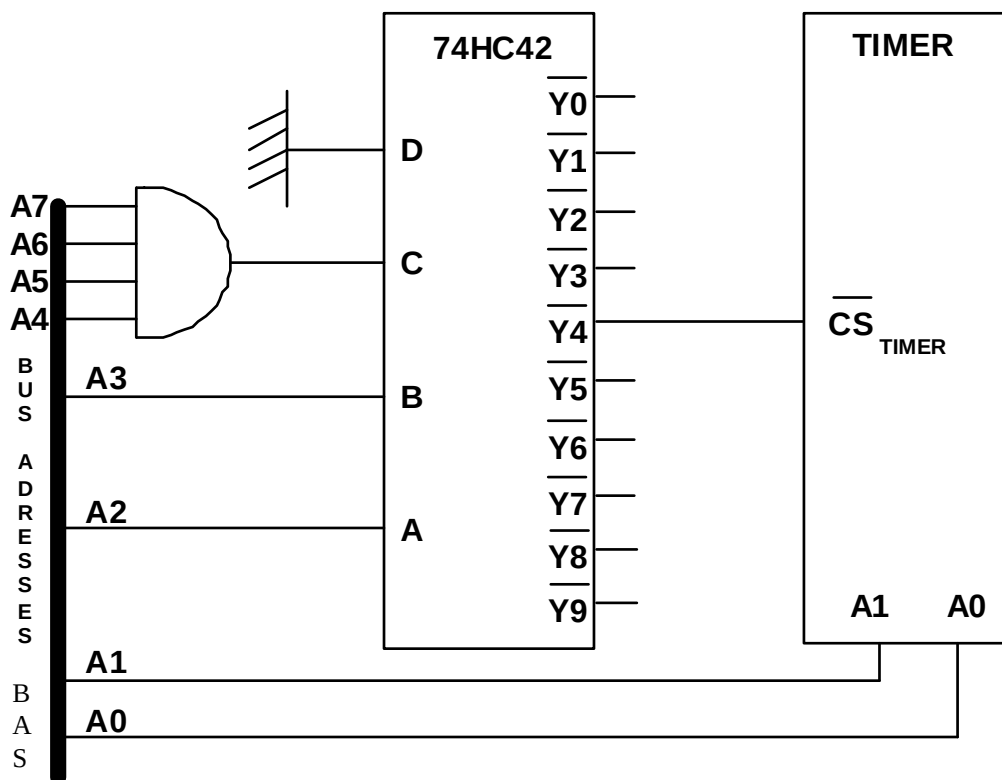
Cet exercice permet une mise en pratique des **notions fondamentales** vues en cours.

On a isolé certaines parties d'un schéma électronique plus complet, parties correspondant au décodage d'adresses de certains composants utilisés pour cette application.

1. Rappeler l'utilité du **décodage d'adresses** sur un système architecturé autour d'un microprocesseur / microcontrôleur.

2. Le schéma ci-dessous représente le décodage d'adresses permettant d'implanter un composant périphérique type « **timer** ». Toutes les broches de ce circuit n'ont pas été représentées sur ce schéma, seules celles ayant un rapport direct avec la fonction décodage d'adresses l'ont été.

2.1. En utilisant la documentation du **74 HC 42** (et après avoir bien compris sa « truth table »), trouver, dans le mapping, les **adresses absolues** correspondant à ce circuit timer.



2.2. Donner le **nombre de registres internes** accessibles dans le timer.

2.3. Existe-t-il des adresses **fantômes** (ghost) permettant d'accéder au timer ?

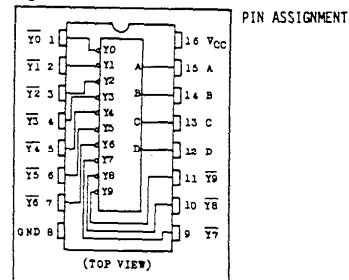
(Pour répondre à cette question, on supposera que bus d'adresse complet est sur 16 bits, A0 à A15, comme le montre la figure suivante).

EXTRAIT DE LA DOCUMENTATION DU 74HC42

TOSHIBA

TC74HC42P BCD-TO-DECIMAL DECODER

The TC74HC42 is a high speed CMOS BCD-TO-DECIMAL DECODER fabricated with silicon gate CMOS technology. It achieves the high speed operation similar to equivalent LSTTL while maintaining CMOS low power dissipation. A BCD code applied to the four inputs (A – D) provides a low level at the selected one of ten decimal decoded outputs. An illegal BCD code such as eleven thru fifteen gives a high level at all outputs. This device also can be used as 3-to-8 LINE DECODER, when D input is assigned as a disable input. This device is useful for code conversion, address decoding, memory selection, demultiplexing, or readout decoding. All inputs are equipped with protection circuits against static discharge or transient excess voltage.



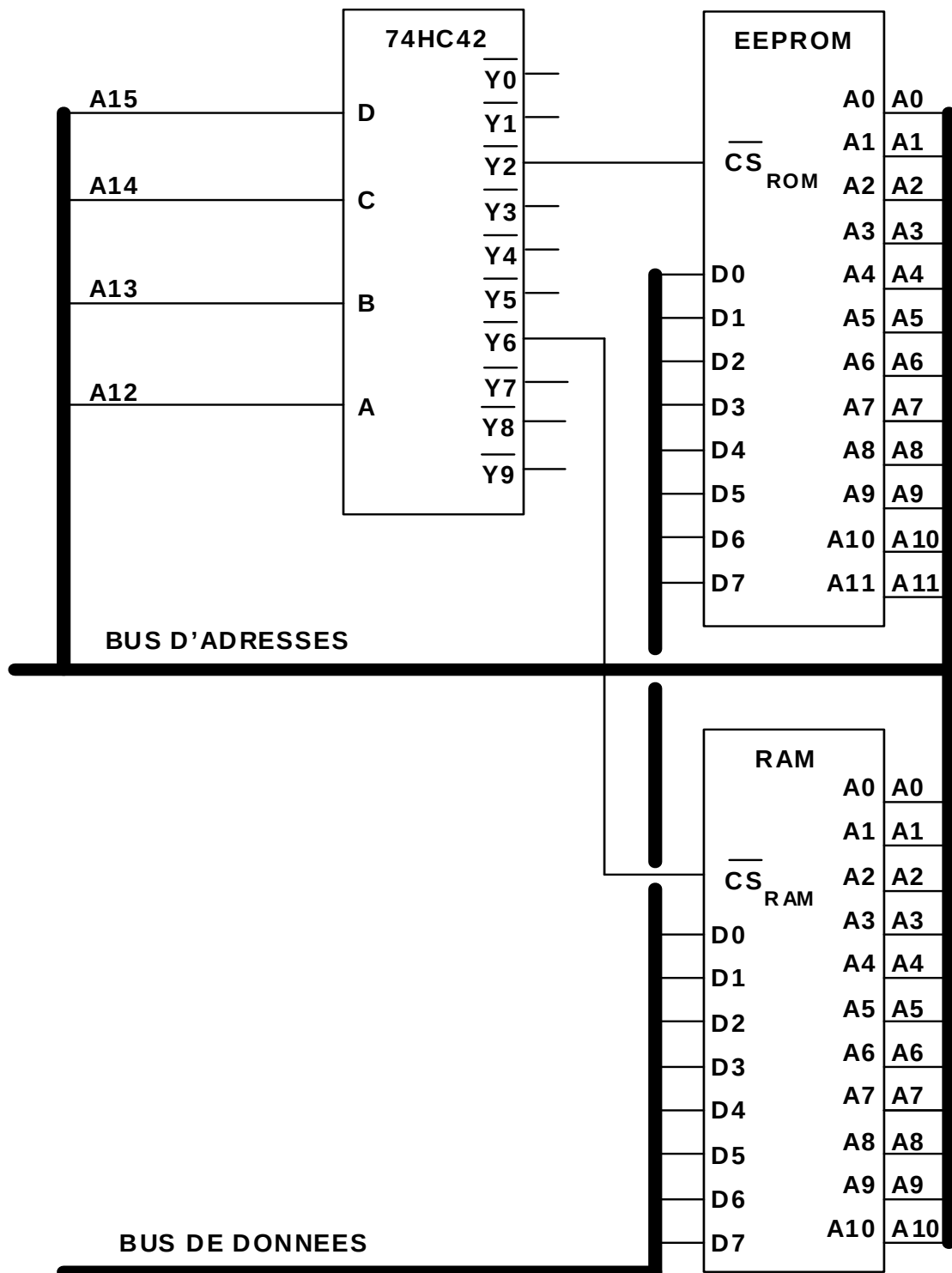
TRUTH TABLE

CODE No.	BCD INPUTS				DECIMAL OUTPUTS									
	D	C	B	A	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇	Y ₈	Y ₉
0	L	L	L	L	L	H	H	H	H	H	H	H	H	H
1	L	L	L	H	H	L	H	H	H	H	H	H	H	H
2	L	L	H	L	H	H	L	H	H	H	H	H	H	H
3	L	L	H	H	H	H	H	L	H	H	H	H	H	H
4	L	H	L	L	H	H	H	H	L	H	H	H	H	H
5	L	H	L	H	H	H	H	H	H	L	H	H	H	H
6	L	H	H	L	H	H	H	H	H	H	L	H	H	H
7	L	H	H	H	H	H	H	H	H	H	H	L	H	H
8	H	L	L	L	H	H	H	H	H	H	H	H	L	H
9	H	L	L	H	H	H	H	H	H	H	H	H	H	L
10	H	L	H	L	H	H	H	H	H	H	H	H	H	H
11	H	L	H	H	H	H	H	H	H	H	H	H	H	H
12	H	H	L	L	H	H	H	H	H	H	H	H	H	H
13	H	H	L	H	H	H	H	H	H	H	H	H	H	H
14	H	H	H	L	H	H	H	H	H	H	H	H	H	H
15	H	H	H	H	H	H	H	H	H	H	H	H	H	H

AC ELECTRICAL CHARACTERISTICS (C_L = 50 pF, Input t_r = t_f = 6 ns)

PARAMETER	SYMBOL	V _{CC}	T _a = 25 °C			T _a = -40 ~ 85 °C		UNIT
			MIN.	TYP.	MAX.	MIN.	MAX.	
Propagation Delay Time	t _{PLH}	2.0	—	80	165	—	200	ns
	t _{PHL}	4.5	—	21	33	—	40	
		6.0	—	18	28	—	34	
Input Capacitance	C _{IN}		—	5	10	—	10	pF
Power Dissipation Capacitance	C _{PD}		—	67	—	—	—	pF

3. Le schéma ci-dessous représente maintenant le décodage d'adresses permettant d'implanter une **EEPROM** et une **RAM** dans ce système.



3.1. Donner les **capacités** de l'**EEPROM** et de la **RAM**.

3.2. Donner, dans le mapping du système, la **plage d'adresses** pour accéder à l'**EEPROM**.

3.3. Donner, dans le mapping du système, la **plage d'adresses** pour accéder à la **RAM**.

4. ETUDE D'UNE CARTE MICROCONTRÔLEUR.

1. Quelques questions de cours.

1.1. Rappeler les **différents registres de la CPU** (Central Process Unit) du HC11.

1.2. Indiquer **en quelques lignes** :

- le rôle du pointeur de programme **PC**,
- le rôle du **CCR** (Code Condition Register).

1.3. De façon générale, peut-on implanter une pile (**Stack**) en **EEPROM** ?

Si oui, pourquoi ? Si non, pourquoi ?

On donne sur les deux pages suivantes un schéma d'une carte utilisant un **HC11**.

Ce schéma appelle les commentaires généraux suivants :

E = **sortie** du HC11, horloge fonctionnant à une fréquence 4 fois plus faible que celle du quartz à 8 MHz du système,

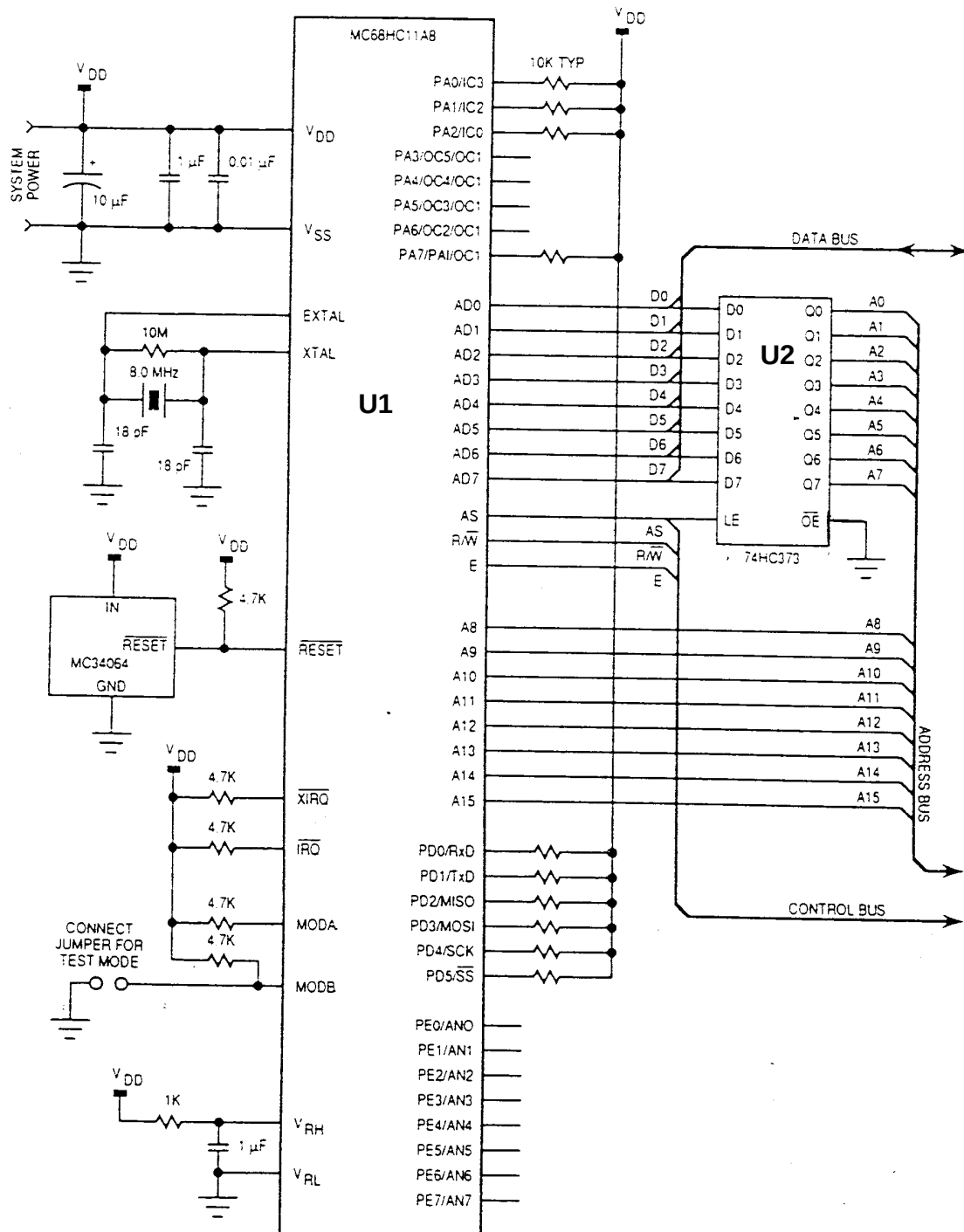
R / $\overline{W}$ = **sortie** du HC11, permettant de différencier les phases de lecture et d'écriture en mémoire externe (niveau **haut** : **lecture** (**read**), niveau **bas** : **écriture** (**write**)),

AD0 .. AD7 = partie basse du bus d'adresses / données multiplexé (tantôt les adresses, tantôt les données) du HC11.

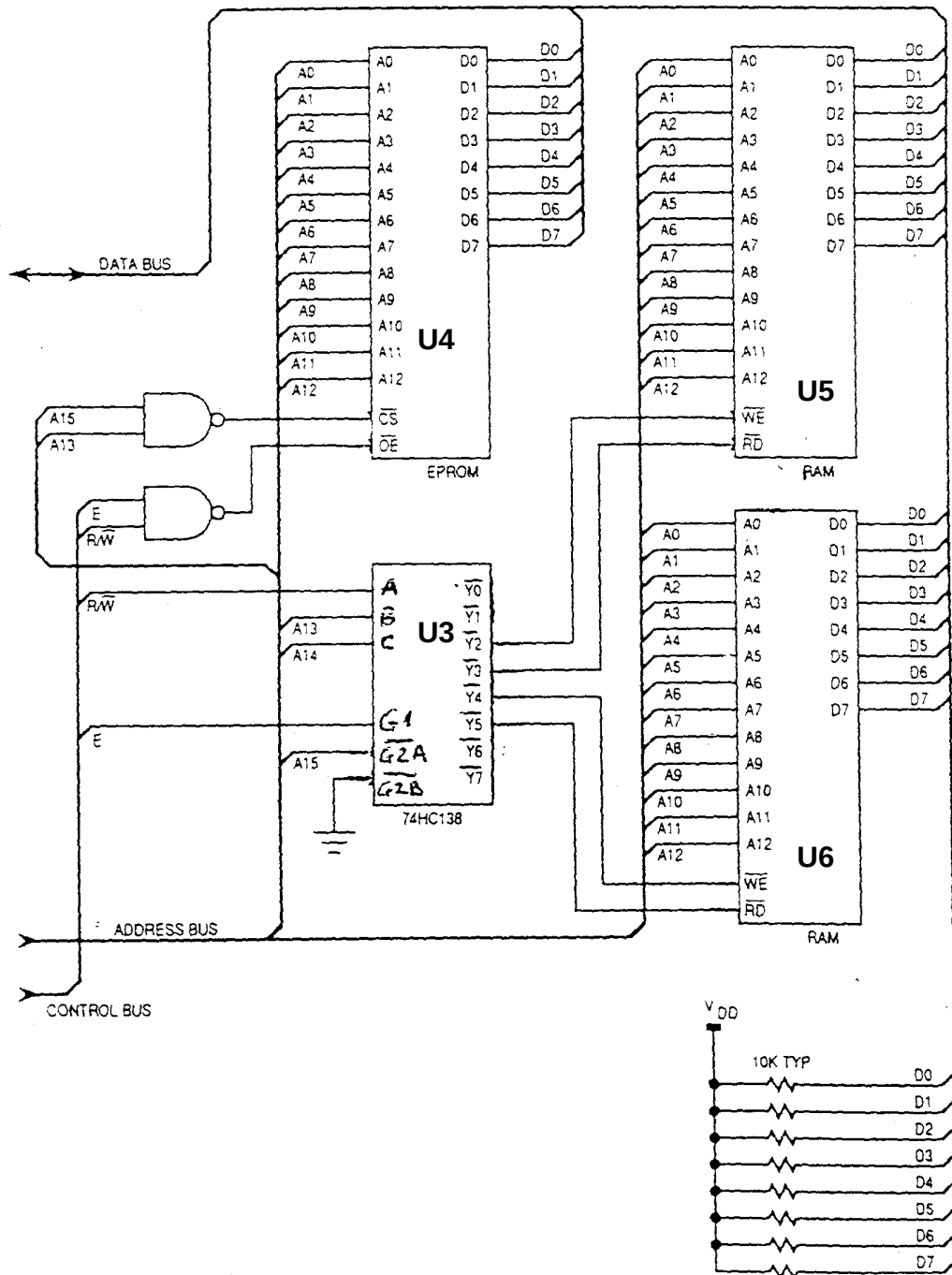
AS = **sortie** du HC11, indiquant une adresse stable sur le bus AD0 .. AD7, les adresses devant être mémorisées sur le font **descendant** de AS,

Le but de cet exercice est de comprendre quelques points essentiels de ce schéma et d'en comprendre les grands principes de fonctionnement.

PARTIE MICROCONTRÔLEUR



PARTIE MEMOIRES



2. Lecture de schéma et repérage des composants.

Remplir le tableau ci-dessous en indiquant pour chacun des composants **Ui** utilisé, sa référence et sa description essentielle en quelques lignes.

Certains de ces composants ont leur documentation constructeur fournie dans les pages suivantes.

<i>Composant</i>	<i>Référence</i>	<i>Description essentielle</i>
<i>U1</i>		
<i>U2</i>		
<i>U3</i>		
<i>U4</i>		
<i>U5</i>		
<i>U6</i>		

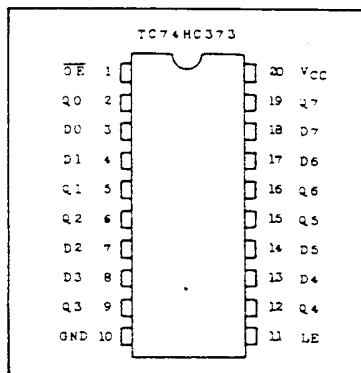
EXTRAIT DE LA DOCUMENTATION DU 74HC373 (référence U2)

TC74HC373P NON-INVERTING OCTAL D-TYPE LATCH WITH 3-STATE OUTPUT

The TC74HC373, TC74HC533 (see page 112), TC74HC563 (see page 114) and TC74HC573 (see page 118) are high speed CMOS OCTAL LATCH with 3-STATE OUTPUT fabricated with silicon gate CMOS technology. These ICs achieve the high speed operation similar to equivalent LSTTL while maintaining the CMOS low power dissipation. These 8-bit D-type latches are controlled by a latch enable input (LE) and an output enable input ($\overline{OE}$).

While the LE input is held in high level, the Q outputs will follow the data input precisely or inversely. When the LE is taken low, the Q outputs will be latched precisely or inversely at the logic level of D input data. While the $\overline{OE}$ input is at low level, the eight outputs will be in a normal logic state (high or low logic level) and while high level the outputs will be in a high impedance state. The application designer has a choice of combination of inverting and non-inverting outputs, symmetrical and neighboring input/output pin layout.

The TX74HC373 and the TC74HC573, the TC74HC533 and the TC74HC563 have the same function and the same characteristics respectively, but have the different pin layouts. The three-state output configuration and the wide choice of outline will make the bus-organized system simple. All inputs are equipped with protection circuits against static discharge or transient excess voltage.



PIN ASSIGNMENT (TOP VIEW)

TRUTH TABLE

INPUTS			OUTPUTS	
$\overline{OE}$	LE	D	Q (HC373, HC573)	$\overline{Q}$ (HC533, HC563)
H	X	X	Z	Z
L	L	X	No change *	No change *
L	H	L	L	H
L	H	H	H	L

X: Don't care

Z: High impedance

*: Q/ $\overline{Q}$ outputs are latched at the time when the LE input is taken low logic level.

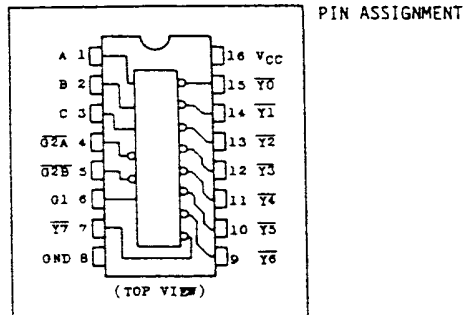
AC ELECTRICAL CHARACTERISTICS ($C_L = 50$ pF, Input $t_r = t_f = 6$ ns)

PARAMETER	SYMBOL	V_{CC}	$T_a = 25^\circ C$			$T_a = -40 \sim 85^\circ C$		UNIT
			MIN.	TYP.	MAX.	MIN.	MAX.	
Propagation Delay Time (D $\rightarrow$ Q, $\overline{Q}$)	t_{PLH}	2.0	—	66	145	—	175	ns
		4.5	—	18	29	—	35	
	t_{PHL}	6.0	—	16	25	—	30	
Input Capacitance	C_{IN}		—	5	10	—	10	pF
Output Capacitance	C_{OUT}		—	10	—	—	—	pF
Power Dissipation Capacitance	C_{PD}		—	41	—	—	—	pF

EXTRAIT DE LA DOCUMENTATION DU 74HC138 (référence U3)

TC74HC138P 3-TO-8 LINE DECODER

The TC74HC138 is a high speed CMOS DECODER (3-8 LINE) fabricated with silicon gate C²MOS technology. It achieves the high speed operation similar to equivalent LSTTL while maintaining the CMOS low power dissipation. If the device is enabled, 3 binary select inputs (A, B and C) determine which one of outputs will go low. Enable input G1 is held "L" level or either G2A or G2B is held "H" level, decoding function is inhibited and all the 8 outputs go high. 3 enable inputs are provided to ease cascade connection and application of address decoder for memory system. All inputs are equipped with protection circuits against static discharge or transient excess voltage.



TRUTH TABLE

INPUTS						OUTPUTS								SELECTED OUTPUT
ENABLE			SELECT			$\bar{Y}_0$	$\bar{Y}_1$	$\bar{Y}_2$	$\bar{Y}_3$	$\bar{Y}_4$	$\bar{Y}_5$	$\bar{Y}_6$	$\bar{Y}_7$	
G1	$\bar{G}2A$	$\bar{G}2B$	C	B	A									
L	X	X	X	X	X	H	H	H	H	H	H	H	H	NONE
X	H	X	X	X	X	H	H	H	H	H	H	H	H	NONE
X	X	H	X	X	X	H	H	H	H	H	H	H	H	NONE
H	L	L	L	L	L	L	H	H	H	H	H	H	H	Y ₀
H	L	L	L	L	H	H	L	H	H	H	H	H	H	Y ₁
H	L	L	L	H	L	H	H	L	H	H	H	H	H	Y ₂
H	L	L	L	H	H	H	H	H	L	H	H	H	H	Y ₃
H	L	L	H	L	L	H	H	H	H	L	H	H	H	Y ₄
H	L	L	H	L	H	H	H	H	H	H	L	H	H	Y ₅
H	L	L	H	H	L	H	H	H	H	H	H	L	H	Y ₆
H	L	L	H	H	H	H	H	H	H	H	H	H	L	Y ₇

X: Don't care

AC ELECTRICAL CHARACTERISTICS ($C_L = 50 \text{ pF}$, Input $t_r = t_f = 6 \text{ ns}$)

PARAMETER	SYMBOL	V_{CC}	$T_a = 25^\circ\text{C}$			$T_a = -40 \sim 85^\circ\text{C}$		UNIT
			MIN.	TYP.	MAX.	MIN.	MAX.	
Propagation Delay Time (A, B, C – $\bar{Y}$)	t_{PLH}	2.0	—	72	175	—	210	ns
	t_{PHL}	4.5	—	23	35	—	42	
		6.0	—	20	30	—	36	
Propagation Delay Time (G, $\bar{G}$ – $\bar{Y}$)	t_{PLH}	2.0	—	62	160	—	195	ns
	t_{PHL}	4.5	—	20	32	—	39	
		6.0	—	17	28	—	34	
Input Capacitance	C_{IN}		—	5	10	—	10	pF
Power Dissipation Capacitance	C_{PD}		—	57	—	—	—	pF

3. Etude de U2.

Expliquer le rôle de **U2** en vous aidant de sa table de vérité fournie précédemment.

4. Capacités mémoire du système.

Donner les **tailles** (en Kilo-octets) des composants **U4**, **U5** et **U6**.

5. Etude de U5 et U6.

On admettra que lorsque les entrées $\overline{WE}$ ($\overline{Write Enable}$) et $\overline{RD}$ ($\overline{Read Enable}$) sont toutes les deux au niveau haut, **U5** et **U6** ne sont pas sélectionnés.

Avant toute chose, bien comprendre la « truth table » du 74HC138 **U3**.

Calculer les **plages d'adresses** de **U5** et de **U6**, en différenciant bien la phase d'écriture et celle de lecture.

6. Etude de U4.

Donner le rôle des entrées $\overline{OE}$ ($\overline{Output Enable}$) et $\overline{CS}$ ($\overline{Chip Select}$).

Calculer la **plage d'adresses** de **U4** et mettre en évidence une plage "fantôme".

7. Mapping mémoire.

Donner le **mapping mémoire** (cartographie mémoire) de cette architecture.

8. Implantation d'une pile.

Proposer une place pour implanter la pile sur ce système.

9. Adresse dans le PC.

Que se passerait-il, si à un instant donné, le pointeur de programme **PC** contenait \$7100 ?

TD 2 : ASPECTS LOGICIELS

1. IMPLANTATION MEMOIRE ET MODES D'ADRESSAGE.

On donne le listing source suivant :

```
*****
* définition des constantes
*****
RAM_HC11      EQU      $0
DONNEES       EQU      $2000
PROGRAMME     EQU      $4000
ZERO          EQU      $0
INIT          EQU      $FF

*****
* zone d'implantation des données
*****

CAR           ORG      RAM_HC11
              RMB      1
TIME          FDB      512

MESSAGE       ORG      DONNEES
              FCC      'ELEC2'
              FCB      ZERO
TEMPO         RMB      2

*****
* zone d'implantation du programme
*****
              ORG      PROGRAMME

*****
* ci-dessous quelques instructions pour remplir
* le tableau des modes d'adressage demandé ci-après
*****
              LDD      TIME
              STD      TEMPO
              LDAA     #INIT
              INCA
              LDY      MESSAGE
              LDY      #MESSAGE
              LDAB     2,Y
              STAB     CAR
```

1. Implantation mémoire.

Remplir le **document ci-après** en indiquant **en hexadécimal** le contenu des cases mémoires indiquées lorsque ce bout de listing aura été compilé et téléchargé sur une carte cible. Lorsque les valeurs ne sont pas définies, on indiquera le signe ?

2. Modes d'adressage et contenu des registres internes du HC11.

Remplir le tableau ci-après en indiquant les modes d'adressage utilisés et les contenus des registres ou variables demandés **après** exécution de chaque instruction. **Ces contenus seront exprimés en hexadécimal.**

IMPLANTATION MEMOIRE ET MODES D'ADRESSAGE

IMPLANTATION MEMOIRE

Adresses	Données		Adresses	Données
\$0000		ECRIRE LES DONNEES GLOBALES EN HEXADECIMAL	\$2000	
\$0001			\$2001	
\$0002			\$2002	
\$0003			\$2003	
\$0004			\$2004	
\$0005			\$2005	
\$0006			\$2006	
\$0007			\$2007	

ET MODES D'ADRESSAGE

Instructions	Mode d'adressage utilisé	Contenu des registres ou des variables demandés <u>après</u> exécution de chaque instruction. (en hexadécimal)
LDD TIME		A = B = valeur de TIME = valeur de TEMPO =
STD TEMPO		A = B = valeur de TIME = valeur de TEMPO =
LDAA #INIT		A =
INCA		A =
LDY MESSAGE		Y =
LDY #MESSAGE		Y =
LDAB 2,Y		B =Y = valeur de CAR =
STAB CAR		B =X = valeur de CAR =

3. Nouveau listing.

Cette partie est indépendante des questions précédentes.
On donne maintenant le nouveau listing suivant

```
*****
* definition des constantes
*****
RAM_HC11      EQU      $0
DONNEES       EQU      $2000
PROGRAMME     EQU      $3000
ZERO          EQU      $0

*****
* zone d'implantation des donnees
*****
DUREE         ORG      RAM_HC11
              FDB      $1234          * valeur de test

              ORG      DONNEES
HEURE         FCB      $0
MINUTE        FCB      $0
SECONDE       FCB      $0
              FCB      ZERO

*****
* zone d'implantation du programme
*****
              ORG      PROGRAMME
*****
* ci-dessous quelques instructions pour remplir
* le tableau des modes d'adressage demandé ci-après
*****
              LDAA     SECONDE
              INCA
              LDAB     #42
              STAB     MINUTE
              LDY      #HEURE
              STAA     0,Y
              LDX      #DUREE
              LDX      DUREE
```

Remplir le tableau de la page suivante en indiquant les modes d'adressage utilisés et les contenus des registres ou variables demandés **après** exécution de chaque instruction.

TABLEAU A COMPLETER

Instructions	Mode d'adressage utilisé	Contenu du registre ou de la variable précisée
LDAA SECONDE		A =
INCA		A =
LDAB #42		B =
STAB MINUTE		valeur de MINUTE
LDY #HEURE		Y =
STAA 0,Y		valeur de HEURE =
LDX #DUREE		X =
LDX DUREE		X =

2. ECRITURE DE MODULES EN ASSEMBLEUR.

1. Réaliser un module conversion nommé **CONV_HMS** qui :

- reçoit dans **D** un nombre de secondes, (par exemple ici la valeur de **DUREE** définie précédemment)
- reçoit dans **Y** un pointeur vers la variable **HEURE** (définie précédemment)
- place dans les variables globales **HEURE**, **MINUTE** et **SECONDE** (définies précédemment) la valeur de **DUREE** exprimée dans ces unités. (les résultats seront stockés en hexadécimal).

Ce programme commencera par une séquence du type (et à compléter) :

	LDD		* à compléter
	LDY		* à compléter
	BSR	CONV_HMS	
	SWI		* retour au moniteur
CONV_HMS			* à compléter
			* à compléter
			* à compléter
			* à compléter
			* à compléter
	RTS		

2. Réaliser un module de conversion nommé **MIN_2_MAJ** qui :

- reçoit dans **X** un pointeur vers une chaîne de caractères se terminant par \$00
 - effectue une conversion de minuscule à majuscule sur les caractères de cette chaîne et place le résultat (en majuscules et en lieu et place pour chaque caractère) dans la même chaîne.
- Il faudra penser à gérer le fait que si un caractère est déjà en majuscule, il faut le laisser ainsi. Les caractères sont codés en ASCII 7 bits (c'est-à-dire qu'on ne gèrera pas les problèmes des caractères accentués)

Par exemple, si la chaîne est **MESS_A_CONV** définie par :

MESS_A_CONV	FCC	'ceci Est uN Message a mettre en mAjuscles'
	FCB	0

au retour du module **MIN_2_MAJ**, la chaîne **MESS_A_CONV** vaudra :

'CECI EST UN MESSAGE A METTRE EN MAJUSCULES'

Ce programme commencera par une séquence du type (et à compléter) :

	LDX		* à compléter
	BSR	MIN_MAJ	
	SWI		* retour au moniteur
MIN_2_MAJ			* à compléter
			* à compléter
	.		
	.		
	.		
	.		
	.		
	RTS		

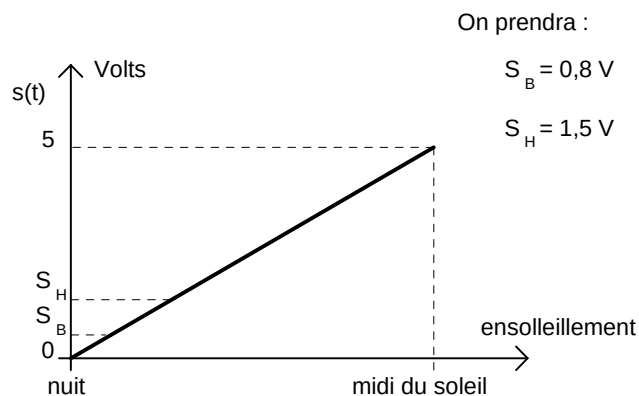
On donnera également au préalable **un organigramme ou une analyse de la solution retenue.**

3. REALISATION D'UNE PETITE APPLICATION.

On se propose ici d'étudier un petit automatisme basé sur un microcontrôleur de type **HC11** permettant de piloter :

- l'ouverture et la fermeture d'un store déroulant,
- l'allumage et l'extinction d'un lampadaire de jardin,

en fonction de la luminosité ambiante. Ainsi, une cellule photoélectrique fournit une tension **analogique** $s(t)$, comprise entre 0V et 5V, dont la variation en fonction de l'ensoleillement est donnée par la courbe ci-dessous :



On supposera que l'allumage du lampadaire ou une nuit de pleine lune ne font pas passer $s(t)$ au-dessus du seuil S_H .

On souhaite :

- à la tombée de la nuit, **allumer** le lampadaire de jardin et **fermer** le store,
- au lever du jour, effectuer les **opérations inverses**.

L'étude se fera en 2 temps :

- analyse **fonctionnelle** générale (partie 1.),
- **écriture** de **certains** modules (partie 2.).

1. Analyse globale.

On suppose disposer des modules logiciels (ou sous-programmes) suivants, identifiés par leur nom :

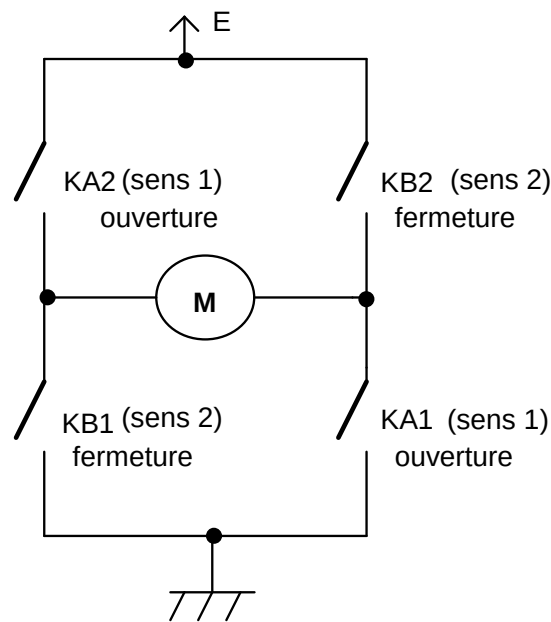
- **OUVIR** : module d'ouverture du store,
- **FERMER** : module de fermeture du store,
- **ENSOLEILLEMENT** : pour acquérir et convertir $s(t)$ en numérique. Ce module renvoie dans l'accumulateur A le résultat de la conversion.
- **ALLUMER** : allumage du lampadaire extérieur,
- **ETEINDRE** : extinction de ce lampadaire.
- **INIT** : différentes initialisations (sens des ports utilisés, mise en route du **CAN** (Convertisseur Analogique / Numérique) ...)

1.2. Ecrire en assembleur le programme correspondant à cet organigramme (on ne détaillera pas encore le contenu des modules mais on se contentera de les appeler par leurs noms définis ci-dessus).

2. Ecriture de certains modules en langage assembleur.

Les volets de la fenêtre sont de type store déroulant. Un moteur à courant continu (associé à un réducteur de vitesse à engrenages) fonctionnant sous une tension nominale E disponible au niveau de la fenêtre permet, en tournant dans le sens 1 de remonter les stores et en tournant dans le sens 2 de les descendre).

Ce moteur est alimenté comme indiqué sur le schéma ci-dessous :



Sur ce schéma les interrupteurs **KAi** et **KBi** supposés idéaux sont pilotés respectivement par des tensions compatibles TTL notées **Ai** et **Bi** avec la logique suivante :

- A_i (ou B_i) au niveau haut: KA_i (ou KB_i) fermé,
- A_i (ou B_i) au niveau bas : KA_i (ou KB_i) ouvert.

Il existe de plus deux capteurs dits de fin de course qui détectent l'ouverture totale et la fermeture totale du store :

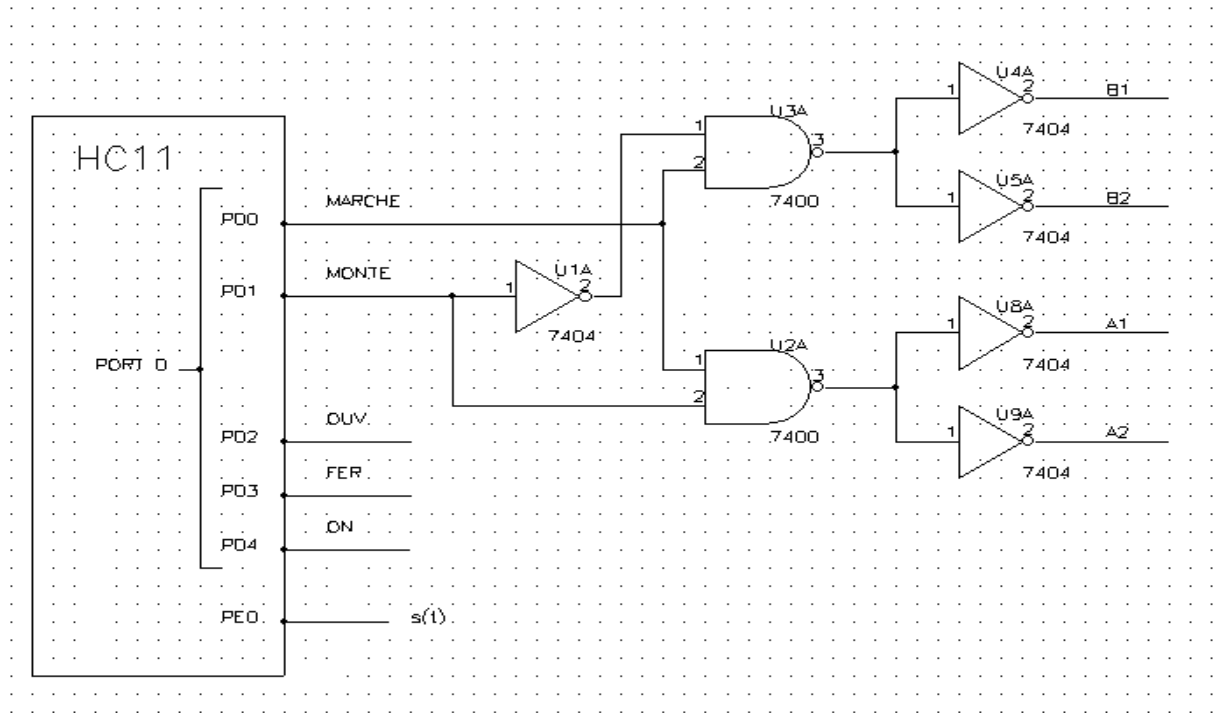
- le capteur d'ouverture génère un niveau TTL noté **OUV** qui est au niveau haut lorsque le store est entièrement **ouvert** (le moteur doit donc être arrêté) et au niveau bas sinon,
- le capteur de fermeture génère un niveau TTL noté **FER** qui est au niveau haut lorsque le store est entièrement **fermé** (le moteur doit donc être arrêté) et au niveau bas sinon.

D'autre part, la commande du lampadaire se fait par une variable logique TTL, notée **ON** qui :

- au niveau haut, permet d'allumer le lampadaire,
- au niveau bas de l'éteindre.

Le schéma électronique concernant ces entrées / sorties **TOR** (**Tout Ou Rien**, c'est-à-dire 0V ou 5V) et leur implantation sur le HC11 est fourni à la page suivante.

On suppose enfin que la cellule photovoltaïque est branchée sur l'entrée **PE0** du Convertisseur Analogique / Numérique (**CAN** ou **ADC**) du HC11.



On suppose que dans le système utilisé, les variables seront stockées à partir de l'adresse **\$2000** et que le programme débutera à l'adresse **\$3000**.

On insistera beaucoup sur la lisibilité du source. Lorsque des variables ou des constantes seront introduites, il faudra les déclarer par les directives d'assemblage adaptées (type EQU, RMB ou autres ...). En particulier, on ne mettra pas dans le code des valeurs directement écrites en hexadécimal (ou décimal, ou binaire), là où une variable ou une constante aurait pu être introduite. Les commentaires seront aussi très appréciés.

2.1. Module INIT.

Ecrire le module **INIT** :

- qui configure correctement la direction des ports utilisés,
- qui met en route le CAN.

On utilisera l'**adressage indexé par X** qui aura été initialisé correctement par la constante :

BASE_REGS EQU \$1000.

2.2. Module ALLUMER.

Ecrire en assembleur le module **ALLUMER**. (on utilisera aussi l'adressage indexé par X).

2.3. Module OUVRIR.

Montrer que l'introduction des variables logiques **MARCHE** et **MONTE**, permet, en les commandant correctement, de générer les signaux **Ai** et **Bi** de pilotage du moteur des stores.

Ecrire en assembleur le module **OUVRIR**.

Attention : ne pas oublier de tester l'état du capteur détectant l'ouverture totale du store ...

2.4. Module ENSOLEILLEMENT.

Donner un **organigramme** possible pour le module **ENSOLEILLEMENT**.

Ecrire ensuite en **assembleur** ce module qui renvoie dans l'accumulateur A la valeur numérique de s(t).